



universität
wien

052400-1 VU Information Management and Systems Engineering (2025S)

Milestone 2

Group 05

Student 1: Golovanov, Ilja, 12133820

Student 2: Togizbayev, Yernaz, 01429473

[23.08.2026], Vienna

Contents

1	Milestone 1 - Recap and Revisions	3
1.1	Recap: ER-Diagram	3
1.2	Revisions and Changes	3
2	Milestone 2	4
2.1	Infrastructure	4
2.2	RDBMS Implementation	4
2.3	NoSQL Implementation	7

1. Milestone 1 - Recap and Revisions

1.1. Recap: ER-Diagram

The conceptual model is unchanged in structure since Milestone 1. Nine entities, one weak entity, one unary relationship, two m:n relationships and two IS-A hierarchies. The attributes listed in Section 1.2 were added during implementation; no entity or relationship was added or removed.

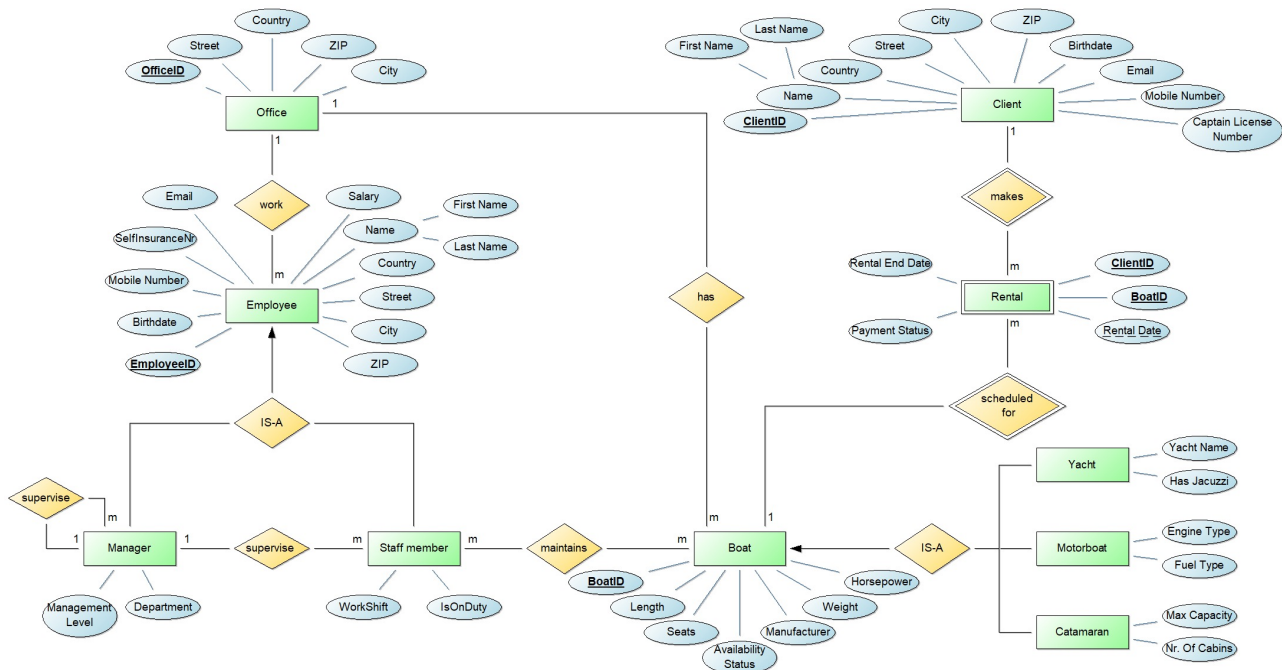


Figure 1: Boat Rental ER diagram (Chen notation)

1.2. Revisions and Changes

No entities or relationships changed. Seven attributes were added, each of them because the implementation needed a fact the model did not record:

- **Boat.DailyRate** DECIMAL(10,2). A rental business without a price is not one. Nullable, because a manager can add a boat before deciding what to charge for it; an unpriced boat is hidden from clients rather than offered at nothing.
- **Rental.TotalAmount** DECIMAL(10,2). What the client was charged, frozen when the booking is made. Recomputing it from DailyRate would let a later price change rewrite what somebody already paid.
- **Rental.StartTime** TIME. A charter starts at a handover time, 09:00 by default. Kept separate from RentalDate rather than widening it to a datetime, because the composite primary key, the rental URLs and the overlap check all key off a date.
- **Rental.CreatedAt** DATETIME. An unpaid booking is a hold with a deadline, and the deadline is measured from when the booking was made. Without it, a booking made a minute ago is indistinguishable from one that lapsed overnight.
- **Rental.CancelledAt** and **Rental.RefundedAt** DATETIME. A paid charter that is called off keeps its row, because that row is the only record the client was charged. These two say when the refund became owed and when it was returned.

- **Catamaran.HasJacuzzi** BOOLEAN. The MS1 model gave this to yachts only, but catamarans have them too, and the booking page states the answer either way.

The relational structure itself is otherwise as submitted: `Rental` remains the weak entity with the composite key (`ClientID`, `BoatID`, `RentalDate`), `Manager.SupervisorID` remains the unary relationship, and `Supervises` and `Maintains` remain the two m:n tables.

2. Milestone 2

2.1. Infrastructure

Three containers, defined in `docker-compose.yml` and started together:

- **db** – MariaDB 11.3. The relational database and the system of record.
- **mongo** – MongoDB 7. The document database the data is migrated into.
- **backend** – Python 3.11 with Flask, serving both the pages and the application logic. It waits for both databases to report healthy before starting.

All dependencies are installed during the image build from `backend/requirements.txt`; nothing has to be installed on the host.

Running it

```
docker compose up --build
```

The application is then at <http://localhost:5000>. Both published ports are bound to `127.0.0.1` (the app on 5000, MariaDB on 3306, MongoDB on 27017), so nothing is reachable from outside the machine running it: the Flask development server would otherwise put a Python traceback in front of anyone on the same network, and the databases would answer with credentials that are in the compose file.

To stop and reset the relational database completely:

```
docker compose down -v
```

The `-v` is required to drop the volume, which is what makes the SQL init scripts run again on the next start.

Signing in

There is no password system. `/login` lists every client and one click signs you in; `/manager/login` picks a manager from a dropdown. Both roles are needed to see the whole application: the booking flow is a client's, and the fleet, staff and MongoDB pages are a manager's.

2.2. RDBMS Implementation

2.2.1. DB Setup / Data Import / Base Function of Web System

The tables are created by the SQL in `database/`, not by the application. MariaDB's entrypoint runs `Group05_Createtable.sql` on first start, followed by `init.sql`, which sources the per-student insert scripts. The entrypoint does not recurse into subdirectories, which is why `init.sql` exists at the top level purely to `SOURCE` the files in `Student1/` and `Student2/`.

The application layer mirrors that schema in `backend/boat_rental/models.py` as SQLAlchemy models. The two are maintained by hand and have to agree: there is no migration tool, so a column added to one must be added to the other.

Data import

backend/boat_rental/generator.py fills the database with randomised data: 200 boats spread over every office, ten clients, employees split between managers and staff, and roughly 340 rentals. It is triggered by the **Generate demo data** button, which is on every page and does not require signing in, so a fresh checkout can be populated before anyone has logged in.

Two details are deliberate. The generator replaces all boats, people and rentals, but it does *not* delete the offices: a manager can open an office in any city, and clearing them would make the only restocking tool destroy every city anyone had added. It also runs as a single transaction, so a failure halfway through cannot leave the database emptied but not refilled.

Boats are generated coherently rather than at random: length is drawn per boat type, and seats, engine power, weight, cabins and the daily rate are derived from it, so the fleet does not contain a seven-metre yacht that sleeps five.

Base functionality

The web system covers client registration and sign-in, harbour and date search, booking with a simulated card payment, cancellation, and a client logbook. On the manager side it covers full CRUD for boats, offices and employees, the two m:n assignment relations, cancellation of any client's charter, and the MongoDB migration and reports described in Section 2.3.

Student 1 (Ilja Golovanov)

2.2.2. Implementation of the Use Case and the Analytics Report

Use case

A client books a boat. They choose a harbour and a date range; the system shows the boats in that harbour, and the client picks one and pays for the charter.

The whole fleet of that harbour is listed rather than only what is free. Boats that are already booked or under maintenance are shown greyed out, labelled with the reason, and carry no radio button at all, so they cannot be selected or submitted. A page showing three boats reads as a company with three boats; showing the taken ones tells the client the harbour is worth trying on other dates.

Analytics report

Which boats are free in a given harbour over a given date range, and how much of the fleet that represents. It is the **Availability** page. Alongside the list it shows the fleet size and how many boats are not under maintenance, so the free count has something to be measured against.

How the report changes

Booking a boat removes it from the report for any overlapping date range. The report and the booking page share one overlap predicate in the code, so what the report calls free and what the booking form accepts cannot disagree.

Changes since MS1

Two conditions were added to the query as the implementation exposed them. A boat with no `DailyRate` is excluded, since it would otherwise reach the checkout with a total of nothing. A cancelled rental no longer counts as an overlap, since its row is kept as a payment record but the boat is free again. The search parameters were narrowed to harbour and dates; skipper and boat type were not implemented.

Student 2 (Yernaz Togizbayev)

2.2.2. Implementation of the Use Case and the Analytics Report

Use case

A manager hires a staff member. The manager fills in the employee's personal and work details and chooses the role. The application writes the `Employee` row and then the matching `Staff` or `Manager` row, and the new staff member is placed under a manager's supervision through the `Supervises` relation.

This exercises the IS-A hierarchy directly. Changing an employee's role later moves them between the `Staff` and `Manager` tables, deleting the old subclass row and writing the new one, rather than trying to update across them.

Analytics report

Which staff a given manager supervises, with each one's shift, duty status and office. It is the **Supervision** page. The SQL joins `Supervises` to `Staff`, to `Employee` twice (once for the staff member and once for the manager), to `Manager` and to `Office`.

How the report changes

Hiring a staff member and assigning them to a manager adds a row to that manager's report. Measured on the running system: before the use case, manager E8 supervised three staff; after hiring Lena Karras and assigning her, the same report returned four.

Changes since MS1

The use case is as designed. One thing the implementation forced: removing an employee is not a single delete. Three foreign key families point at them – `Manager.SupervisorID`, `Supervises` and `Maintains` – and all three have to be detached first, including when a role change deletes the old subclass row. The MS1 design did not account for that.

2.3. NoSQL Implementation

2.3.1. NoSQL Re-Design

The relational schema spreads the application over ten tables. Answering either use case there means joining four or five of them. The document model instead stores each use case's answer in the shape it is asked for, in three collections.

offices

One document per harbour, with its boats embedded, and each boat's rentals embedded inside the boat. Student 1's report becomes a single aggregation over one collection.

```
{
  "_id": "05",
  "city": "Dubrovnik",
  "country": "Croatia",
  "street": "Obala Stjepana Radica, 2",
  "zip": "20000",
  "boats": [
    {
      "boatId": "B27",
      "manufacturer": "Lagoon",
      "type": "catamaran",
      "length": 15.2,
      "seats": 10,
      "availabilityStatus": "Available",
      "dailyRate": "1120.00",
      "jacuzzi": true,
      "rentals": [
        { "clientId": "C3", "boatId": "B27",
          "rentalDate": "2026-09-04", "rentalEndDate": "2026-09-09",
          "paymentStatus": "PAID", "totalAmount": "5600.00" }
      ]
    }
  ]
}
```

managers

One document per manager, with the staff they supervise embedded. Student 2's report becomes a single document read.

```
{
  "_id": "E8",
  "employeeId": "E8",
  "name": { "first": "Jamie", "last": "Miller" },
  "email": "jamie.miller@boat-rental.com",
  "office": { "officeId": "01", "city": "Mykonos", "country": "Greece" },
  "department": "HR",
  "managementLevel": "Senior",
  "supervisorId": null,
  "supervisedStaff": [
    { "employeeId": "E2",
      "name": { "first": "Avery", "last": "Smith" },
      "email": "avery.smith@boat-rental.com",
      "office": { "officeId": "01", "city": "Mykonos", "country": "Greece" },
      "workShift": "Evening", "isOnDuty": true }
  ]
}
```

clients

One document per client with their own rentals embedded, which is the client logbook.

Keys, and what happened to them

Primary keys are kept, as `_id`: `OfficeID`, `ManagerID` and `ClientID` are meaningful identifiers that the application already uses in URLs, so replacing them with generated `ObjectIds` would add a lookup and gain nothing. Foreign keys mostly disappear: `Boat.OfficeID` is implied by the boat being inside its office document, and `Supervises` stops being a table at all because the relationship is the array. The keys that survive are the ones crossing a document boundary. A rental keeps `clientId` and `boatId`, because it is embedded in two places and each copy needs to name the other side.

The IS-A hierarchies are flattened. `Employee` is folded into each manager and staff sub-document, and the boat subtypes become a `type` field plus the attributes that subtype has. Nothing in the application asks for "an employee" or "a boat" without already knowing which kind it is, so the parent table earns nothing as a separate document.

Changes compared to the individual MS1 designs

Student 1's MS1 design embedded rentals in the client. Student 2's embedded supervised staff in the manager. Both survive, but neither on its own serves the other's use case, so the team design keeps both and adds the `offices` collection with boats embedded. That is what makes Student 1's report a single-collection query: his MS1 design would have needed a scan of every client's rentals to find out whether a boat was free.

The cost is that a rental is stored twice, once under its boat and once under its client. That is the trade the document model asks for here: both reads become single-document, and the duplication costs nothing at runtime because only the migration writes to MongoDB.

2.3.2. NoSQL DB Setup and Data Migration

MongoDB 7 runs as the `mongo` service, with a named volume for its data and a healthcheck the backend waits on. The application talks to it with `pymongo`; all the MongoDB code is in `backend/boat_rental/nosql.py`.

The migration is started from the **MongoDB** page in the manager navigation, with the **Migrate to MongoDB** button. It clears all three collections before writing, so running it twice produces the same result rather than duplicating anything.

It is a one-way copy. MariaDB stays the system of record and nothing is written back to it, and the application never writes to both databases at once: normal use writes only to MariaDB, and MongoDB is refreshed by re-running the migration. The data is read through the existing SQLAlchemy models rather than with fresh SQL, so there is a single definition of what a boat or a rental is.

Rentals are grouped by boat and by client in one pass before the documents are assembled, so the migration reads each table once rather than querying per document. On the current demo data one run migrates:

Collection / embedded	Documents
offices	20
managers	4
clients	11
boats (embedded in offices)	200
rentals (embedded, twice each)	323

Semantics are preserved rather than approximated. Money is written as a string, not a double: the columns are `DECIMAL(10, 2)` and a float cannot hold cents exactly, so the same reasoning that chose the SQL type chooses the BSON one. Dates are ISO-8601 strings, which compare correctly as strings and so can be used directly in the range conditions of the availability query.

Student 1 (Ilja Golovanov)

2.3.3. NoSQL Use Case and Analytics Report

The use case and the report were reimplemented against MongoDB and are on the same **MongoDB** page. Selecting a harbour and a date range runs the aggregation below and lists the boats it returns.

The results match the relational implementation. This was checked rather than assumed: for all 20 harbours in the demo data, over the same date range, the SQL query and the aggregation return exactly the same set of boat identifiers. The same holds after the use case is triggered: booking a boat in Dubrovnik for a window that previously had ten free boats leaves nine, and the booked boat is the one missing from the list.

2.3.4. NoSQL Analytics Report Statement

```
db.offices.aggregate([
  { $match: { city: "Dubrovnik" } },
  { $unwind: "$boats" },
  { $match: { "boats.availabilityStatus": "Available",
    "boats.dailyRate": { $ne: null } } },
  { $addFields: {
    clashes: { $filter: {
      input: "$boats.rentals", as: "r",
      cond: { $and: [
        { $lt: [ "$r.rentalDate", "2026-08-30" ] },
        { $or: [ { $eq: [ "$r.rentalEndDate", null ] },
          { $gt: [ "$r.rentalEndDate", "2026-08-23" ] } ] },
        { $ne: [ "$r.paymentStatus", "CANCELLED" ] } ] } } } },
    { $match: { clashes: { $size: 0 } } },
    { $project: { _id: 0, boatId: "$boats.boatId",
      manufacturer: "$boats.manufacturer", type: "$boats.type",
      seats: "$boats.seats", length: "$boats.length",
      jacuzzi: "$boats.jacuzzi", dailyRate: "$boats.dailyRate",
      city: "$city", country: "$country" } },
    { $sort: { manufacturer: 1, boatId: 1 } }
  ]})
```

How the design affects it

The whole query stays inside one collection. The relational version joins Boat to Office and then tests a correlated NOT EXISTS against Rental; here the harbour is selected by an indexed equality, and the rentals that would clash are already inside the boat. The overlap test itself is unchanged in meaning, it has just moved from a subquery into a \$filter over an embedded array.

Compromises

Two, both accepted knowingly. First, the same rental document exists under the boat and under the client, so a booking would have to be written twice. That is tolerable here because nothing but the migration writes to MongoDB, but it would be the first thing to revisit if the application started writing to the document database directly. Second, \$unwind expands one document per boat, so a harbour with a very large fleet does more work in memory than a row-per-boat table would. The mitigation is that the \$match on city runs first and is index-backed, so only the selected harbour is ever unwound. A harbour holding thousands of boats would push the document toward MongoDB's 16 MB limit and would be the point to switch the boats to their own collection with a reference back to the office.

2.3.5. NoSQL Indexing

The report selects a harbour by name, so offices.city is indexed:

```
db.offices.createIndex({ city: 1 }, { name: "city_idx" })
```

Measured with explain("executionStats") on the demo data, running the aggregation above with the index dropped and then in place:

Index	Docs examined	Keys examined	Returned	Plan
dropped	20	0	1	COLLSCAN
in place	1	1	1	IXSCAN

Without the index every harbour document is fetched and discarded; with it, exactly the one needed is read. The ratio is the number of harbours, so it grows as the company opens offices. The absolute times are both around a millisecond at this data size, which is the honest result: the index matters for the number of documents touched, not for wall-clock time on 20 documents.

Student 2 (Yernaz Togizbayev)

2.3.3. NoSQL Use Case and Analytics Report

The use case writes to MariaDB, and the report was reimplemented against MongoDB on the **MongoDB** page: selecting a manager shows the staff that manager supervises, with shift, duty status and office.

The results match the relational implementation, checked across all managers that supervise anyone: the same staff identifiers come back from both databases. The report also changes with the use case in the same way. Manager E8 supervised three staff; after hiring Lena Karras, assigning her to E8 and re-running the migration, the MongoDB report returned four, exactly as the SQL report did.

2.3.4. NoSQL Analytics Report Statement

```
db.managers.find({ _id: "E8" })
```

How the design affects it

This is the clearest gain of the two. The relational report joins five tables: Supervises, Staff, Employee (twice), Manager and Office. Embedding the supervised staff inside the manager reduces it to fetching one document by its primary key, which MongoDB serves from the `_id` index with no additional index required. Each staff sub-document already carries the name, shift, duty flag and office, so there is nothing left to look up afterwards.

Compromises

Embedding means staff details are duplicated wherever a person appears, so a change of office or shift has to be written in more than one place. That is acceptable in this application because employee details change rarely and the report is read often, and because the migration rebuilds the documents wholesale rather than patching them.

The real cost is directional. The design answers "who does this manager supervise" in one read, but the reverse question, "who supervises this staff member", has no natural home: without help it means scanning every manager document and searching its array. That is what the index in the next section exists for. A design that instead kept staff in their own collection with a `managerId` reference would make the reverse question trivial and the report a join, which is the wrong trade for this use case, since the report is what the application actually runs.

2.3.5. NoSQL Indexing

The report itself needs no new index, because it is a lookup on `_id`. The reverse lookup does, so the array field is indexed. MongoDB indexes each element of an embedded array separately, which is what makes this work at all:

```
db.managers.createIndex({ "supervisedStaff.employeeId": 1 },  
  { name: "supervised_staff_idx" })
```

Measured with `explain("executionStats")` on the reverse lookup:

```
db.managers.find({ "supervisedStaff.employeeId": "E2" })
```

Index	Docs examined	Keys examined	Returned	Plan
dropped	4	0	1	COLLSCAN
in place	1	1	1	IXSCAN + FETCH

The collection scan reads every manager document and searches each `supervisedStaff` array; the index goes straight to the one that contains the staff member. With four managers in the demo data the saving is small in absolute terms, and it would be dishonest to present it as a speed-up at this size. What the numbers show is the shape of the change: the scan is proportional to the number of managers, the indexed lookup is not.

Writing this section also caught a real defect. The index was first created on `supervisedStaff.staffId`, but the migration writes the field as `employeeId`, so the index matched nothing and `explain` reported `nReturned: 0` with the index apparently in use. The field name was corrected and the figures above are from the corrected version.